



Constrained Pseudorandom Functions, Revisited

Julian Mouthon, Mario Razafinony

Master 1 Cryptologie, Calcul haute-performance et Algorithmique
Sorbonne Université

22 mai 2026

Introduction

Les fonctions pseudo-aléatoires (*Pseudorandom Functions*, PRF) sont des primitives fondamentales en cryptographie moderne. Elles interviennent dans de nombreux protocoles, notamment pour le chiffrement symétrique, l'authentification ou la dérivation de clés. Dans certains contextes, il est nécessaire de déléguer uniquement une partie des capacités d'évaluation d'une PRF sans révéler la clé secrète complète. Ce besoin apparaît notamment dans le contexte de la recherche sur données chiffrées (*Searchable Symmetric Encryption*) et du stockage externalisé sur des serveurs non fiables, où il est nécessaire de contrôler précisément les capacités d'évaluation d'un adversaire [1]. Les fonctions pseudo-aléatoires contraintes (*Constrained Pseudorandom Functions*, CPRF) répondent à ce problème en permettant de restreindre l'évaluation de la fonction à un sous-ensemble d'entrées défini par une contrainte [1].

Ce projet s'intéresse à la construction de CPRF proposée par Bost, Minaud et Ohrimenko en 2017 [1], basée sur l'itération d'une permutation à trappe. Dans un premier temps, nous présentons les notions fondamentales liées aux fonctions pseudo-aléatoires et aux CPRF, ainsi que la construction proposée dans [1]. Dans un second temps, nous étudions plusieurs variantes de cette construction reposant sur RSA, Rabin, AES, ainsi qu'une alternative basée sur des transformations linéaires, F^{WEAK} , dont nous proposons une version hachée et pour laquelle nous discutons la sécurité. Enfin, nous revenons sur les travaux récents de Cheng et Jaeger [2], qui mettent en évidence une attaque contre cette CPRF et proposent une variante hachée permettant de renforcer sa sécurité.

L'ensemble du projet est accompagné d'une implémentation complète et de simulations permettant d'évaluer expérimentalement les attaques étudiées.

Le code source associé au projet est disponible sur GitHub : <https://github.com/julian-mtn/bmo17-cprf.git>

Table des matières

1 Définitions	3
Fonction pseudo-aléatoire	3
Contrainte	3
Fonction pseudo-aléatoire contrainte	3
2 Construction de la CPRF de BMO17	3
2.1 Propriétés	3
Génération et structure de la clé maîtresse	3
Évaluation de la CPRF avec la clé maîtresse	3
Dérivation et structure de la clé contrainte	4
Évaluation restreinte avec une clé contrainte	4
2.2 CPRF avec la permutation à trappe RSA	4
2.2.1 Implémentation de la permutation à trappe RSA	4
2.2.2 Implémentation de la clé maîtresse et génération de l'état initial	5
2.2.3 Implémentation de la dérivation des clés contraintes	5
3 Étude d'autres permutations alternatives	5
3.1 Permutation linéaire F^{WEAK}	5
3.1.1 Implémentation de la permutation	5
3.1.2 Implémentation de la clé contraintes	5
3.1.3 Approche d'une attaque à base d'un système linéaire	6
3.1.4 Jeu de sécurité pour l'attaque sur F^{WEAK}	8
3.1.5 Renforcement de F^{WEAK} par hachage	8
3.1.6 Preuve de sécurité de F^{WEAK} hachée	9
3.2 Permutation à trappe de Rabin	10
3.3 Permutation à trappe AES	11
4 L'attaque CJ25	11
4.1 Description générale et intuition de l'attaque	11
4.2 Modélisation du jeu de sécurité et avantage de l'attaquant	12
4.3 Mise en œuvre expérimentale de l'attaque	12
4.3.1 Implémentation de l'oracle de sécurité	13
4.3.2 Implémentation de l'attaquant et stratégie	13
5 Renforcement de la CPRF par hachage	13
5.1 Principe général de la CPRF hachée	13
5.2 Construction explicite de la CPRF hachée de Cheng et Jaeger	13
6 Résultats expérimentaux	14

1 Définitions

Fonction pseudo-aléatoire (PRF)

Une fonction pseudo-aléatoire

$$F : \mathcal{K} \times D \rightarrow R$$

est une fonction telle que, pour une clé secrète K , la fonction $F(K, \cdot)$ est indiscernable d'une fonction réellement aléatoire $G : D \rightarrow R$ pour tout adversaire efficace.

Contrainte

Une contrainte est un prédicat booléen

$$C : D \rightarrow \{0, 1\},$$

qui partitionne le domaine D en deux ensembles :

- les entrées *autorisées*, telles que $C(x) = 1$
- les entrées *interdites* (ou contraintes), telles que $C(x) = 0$

Intuitivement, la contrainte spécifie l'ensemble des entrées sur lesquelles une clé contrainte est autorisée à évaluer la fonction pseudo-aléatoire.

Ici, nous considérons principalement des contraintes de la forme

$$C_n(x) = [x < n],$$

qui autorisent toutes les entrées strictement inférieures à un seuil $n \in \mathbb{N}$.

Fonction pseudo-aléatoire contrainte (CPRF)

Une fonction pseudo-aléatoire contrainte (CPRF) est donnée par quatre algorithmes :

- $\text{Setup}(1^\lambda) \rightarrow K$ (clé maître) : génère la clé secrète principale K .
- $\text{Constrain}(K, C) \rightarrow K_C$ (clé contrainte) : produit une clé spéciale K_C permettant d'évaluer la fonction uniquement sur les entrées autorisées par C .
- $\text{Eval}(K, x) \rightarrow y$: évalue la fonction pseudo-aléatoire sur une entrée x avec la clé maître.
- $\text{EvalC}(K_C, x) \rightarrow y$: évalue la fonction sur l'entrée x avec la clé contrainte K_C .

Une CPRF est correcte si, pour toute clé K , toute contrainte C et toute entrée x telle que $C(x) = 1$, on a

$$\text{EvalC}(K_C, x) = \text{Eval}(K, x).$$

Autrement dit, la clé contrainte permet d'évaluer la PRF exactement sur les entrées autorisées.

2 Construction de la CPRF de BMO17

2.1 Propriétés

La CPRF de BMO17 est basée sur l'itération successive d'une permutation à trappe, c'est-à-dire une fonction bijective facile à calculer mais difficile à inverser sans information secrète. On note cette permutation π .

Génération de la clé maîtresse

La clé maîtresse est composée des éléments suivants :

- $ST_0 \in \mathbb{Z}_N$, un état initial secret choisi aléatoirement
- SK , une clé secrète RSA définissant la permutation à trappe π_{SK}

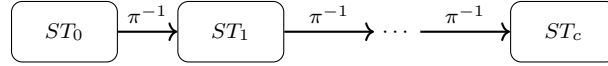
La clé publique associée PK permet uniquement de calculer la permutation directe π .

Évaluation de la CPRF avec la clé maîtresse

Avec la clé maîtresse (ST_0, SK) et une entrée c , on peut évaluer la CPRF sur c par :

$$Eval((SK, ST_0), c) = \pi_{SK}^{-c}(ST_0)$$

L'évaluation consiste à appliquer c fois l'inverse de la permutation à partir de l'état initial ST_0 .

**Génération de la clé contrainte**

À partir de la clé maîtresse (ST_0, SK) et d'un entier n , correspondant à la contrainte $C(c) = [c < n]$, la clé contrainte est composée des éléments suivants :

- PK , la clé publique associée à la permutation π
- $ST_n = \pi_{SK}^{-n}(ST_0)$
- n

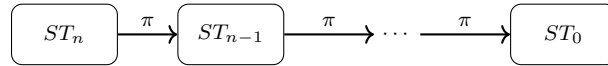
La clé secrète SK n'est pas incluse dans la clé contrainte.

Évaluation de la CPRF avec la clé contrainte

Avec la clé contrainte (PK, ST_n, n) et une entrée c , l'évaluation est possible uniquement si $c < n$. Dans ce cas, la valeur de la CPRF est calculée comme suit :

$$EvalC((PK, ST_n, n), c) = \pi_{PK}^{n-c}(ST_n)$$

Cette valeur est égale à $Eval((SK, ST_0), c)$ par construction, ce qui assure la correction du schéma.

**2.2 CPRF avec la permutation à trappe RSA**

Nous utilisons la bibliothèque OpenSSL pour la gestion des grands entiers (**BIGNUM**), les opérations modulaires et le hachage SHA-256.

L'aléa est sécurisé et est obtenu à partir de `/dev/urandom`, ce qui permet de générer l'état initial ST_0 ainsi que les paramètres RSA de manière plus sûre.

2.2.1 Implémentation de la permutation à trappe RSA

La permutation à trappe utilisée dans BMO17 est défini à l'aide de RSA. Nous implémentons la génération de clés RSA de taille 4096 bits, ainsi que l'évaluation de la permutation dans les deux sens.

Plus précisément, la fonction publique correspond à :

$$x \mapsto x^e \bmod N,$$

tandis que l'inverse de la permutation est calculé via :

$$x \mapsto x^d \bmod N.$$

avec

$$x^{e \cdot d} = x \bmod N, \forall x \in \mathbb{Z}_N.$$

2.2.2 Implémentation de la clé maîtresse et génération de l'état initial

La clé maîtresse de la CPRF est composée d'un état initial secret ST_0 et d'une clé privée RSA. L'état initial est généré comme un entier aléatoire de 256 bits à partir de `/dev/urandom`.

L'évaluation avec la clé maîtresse consiste à appliquer c fois l'inverse de la permutation RSA à partir de ST_0 .

2.2.3 Implémentation de la dérivation des clés contraintes

À partir de la clé maîtresse et d'un paramètre de contrainte n , nous dérivons une clé contrainte composée de la clé publique RSA (e, N) et de l'état

$$ST_n = \pi^{-n}(ST_0).$$

Cette clé permet ensuite d'évaluer la CPRF uniquement pour les entrées $c < n$ en appliquant $(n - c)$ fois la permutation publique à partir de ST_n . La clé privée n'est jamais incluse dans la clé contrainte, ce qui garantit que seules les évaluations autorisées sont possibles.

3 Étude d'autres permutations alternatives

3.1 Permutation linéaire F^{WEAK}

Dans cette section, nous allons voir une implémentation d'une CPRF dont l'entrée et la sortie seront des vecteurs $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ et $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^M$ où p est un nombre premier et N et M les dimensions des vecteurs.

3.1.1 Implémentation de la permutation

Ici, l'évaluation de la permutation à trappe est définie par :

$$Eval(F^{WEAK}, \vec{x}) = Eval(S, \vec{x}) = S \cdot \vec{x}$$

où S est une matrice de taille $M \times N$ à coefficients dans $\mathbb{Z}/p\mathbb{Z}$ où $M \ll N$.

S est la clé maîtresse et elle est choisie aléatoirement lors de la génération de la clé maîtresse.

3.1.2 Implémentation de la clé contraintes

Pour générer la clé contrainte :

- l'attaquant envoie un vecteur $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^N$
- l'oracle tire au hasard un vecteur $\vec{d} \in (\mathbb{Z}/p\mathbb{Z})^M$ tel que $\vec{d} \neq \vec{0}$
- l'oracle renvoie $Constrain(S, \vec{y}) = S_{\vec{y}} = S + d \cdot \vec{y}^T$

En posant

$$CEval(S_{\vec{y}}, \vec{x}) = S_{\vec{y}} \cdot \vec{x}$$

on remarque que l'ensemble des vecteurs contraints sont les $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ tel que $\langle \vec{x}, \vec{y} \rangle = 0$.

En effet,

$$\begin{aligned} CEval(S_{\vec{y}}, \vec{x}) &= S_{\vec{y}} \cdot \vec{x} \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} + d \cdot \vec{y}^T \cdot \vec{x} \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} + d \cdot \langle \vec{x}, \vec{y} \rangle \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} \iff \langle \vec{x}, \vec{y} \rangle = 0 \end{aligned}$$

Donc :

$$S_{\vec{y}} \cdot \vec{x} = \begin{cases} S \cdot \vec{x} & \text{si } \langle \vec{x}, \vec{y} \rangle = 0 \\ \text{résultat 'brouillé'} & \text{sinon} \end{cases}$$

Cette construction permet donc de restreindre l'évaluation de la fonction aux seuls vecteurs orthogonaux à $\vec{y}$. Lorsque l'attaquant envoie un vecteur $\vec{y}$, il est impératif que l'oracle doit refuser si $\vec{y} = \vec{0}$, sinon l'oracle va envoyer $S_{\vec{y}} = S$, ce qui révélera la clé maîtresse à l'attaquant.

3.1.3 Approche d'une attaque à base d'un système linéaire

Notons $\vec{S}_i \in (\mathbb{Z}/p\mathbb{Z})^N$ la i ème ligne de S .

Pour un $\vec{x}$ orthogonal à $\vec{y}$, pour chaque $\vec{S}_i$, notons $k_i = \vec{S}_i \cdot \vec{x} = \vec{S}_{\vec{y}_i} \cdot \vec{x}$.

$$\begin{array}{ccc} S & \vec{x} & \vec{k} \\ \left[\begin{array}{c} \vec{S}_1 \\ \vec{S}_2 \\ \vdots \\ \vec{S}_M \end{array} \right] & \cdot & \left[\begin{array}{c} x_1 \\ x_2 \\ \vdots \\ x_N \end{array} \right] = \left[\begin{array}{c} k_1 \\ k_2 \\ \vdots \\ k_M \end{array} \right] \end{array}$$

FIGURE 1 – Représentation du système linéaire $S \cdot \vec{x} = \vec{k}$

En choisissant N vecteurs $\{\vec{x}_j\}_{1 \leq j \leq N}$ tel que $\langle \vec{x}_j, \vec{y} \rangle = 0, \forall j$, on peut construire la matrice $X \in (\mathbb{Z}/p\mathbb{Z})^{N \times N}$ tel que la j ème ligne de X soit $\vec{x}_j^T$

$$\left[\begin{array}{c} \vec{x}_1 \\ \vec{x}_2 \\ \vdots \\ \vec{x}_N \end{array} \right]$$

FIGURE 2 – Matrice X pour l'attaque construite avec $\{\vec{x}_j\}$ orthogonaux à $\vec{y}$

Pour retrouver la matrice S , il suffit donc de résoudre le système linéaire $X \cdot \vec{S}_i^T = \vec{k}_i$ où le j ème élément de $\vec{k}_i$ est $k_{i,j} = \vec{S}_i \cdot \vec{x}_j$, pour chaque ligne S_i de S .

$$\begin{array}{ccc} X & \vec{S}_i^T & \vec{k}_i \\ \underbrace{\left[\begin{array}{c} \vec{x}_1 \\ \vec{x}_2 \\ \vdots \\ \vec{x}_N \end{array} \right]}_{N \times N} & \cdot \underbrace{\left[\begin{array}{c} S_{i,1} \\ S_{i,2} \\ \vdots \\ S_{i,N} \end{array} \right]}_{N \times 1} = \underbrace{\left[\begin{array}{c} k_{i,1} \\ k_{i,2} \\ \vdots \\ k_{i,N} \end{array} \right]}_{N \times 1} \end{array}$$

FIGURE 3 – Représentation du système linéaire $X \cdot \vec{S}_i^T = \vec{k}_i$, où $k_{i,j} = \vec{S}_i \cdot \vec{x}_j$

On répète M fois ce procédé pour retrouver chacun des $\vec{S}_i$.

Problème : Le système linéaire admet une solution unique si et seulement si la matrice X est inversible, ce qui est équivalent à $\text{rang}(X) = N$. Afin de garantir cette inversibilité, il est nécessaire de choisir des vecteurs $\vec{x}_j$ tels que la famille $\{\vec{x}_j\}_{1 \leq j \leq N}$ soit libre, c'est-à-dire qu'elle engendre un espace vectoriel de dimension N .

Par ailleurs, chaque vecteur $\vec{x}_j$ doit être orthogonal à $\vec{y}$. On a alors que l'ensemble $\{\vec{y}\} \cup \{\vec{x}_j\}_{1 \leq j \leq N}$ est constitué de $N + 1$ vecteurs deux à deux linéairement indépendants, et engendrerait donc un espace de dimension $N + 1$.

Ceci est impossible, puisque ces vecteurs appartiennent tous à $(\mathbb{Z}/p\mathbb{Z})^N$, qui est de dimension N . Cette contradiction montre qu'il est impossible de construire une telle famille de vecteurs $\{\vec{x}_j\}_{1 \leq j \leq N}$ satisfaisant simultanément ces conditions.

Alternative : Dans le cas général, on remarque tout de même que $\vec{y}$ engendre un espace de dimension 1, ce qui implique qu'il est possible de choisir $N - 1$ vecteurs $\{\vec{x}_j\}_{1 \leq j \leq N-1}$ tel que chaque vecteur $\vec{x}_j$ soit orthogonal à $\vec{y}$.

En effet, dans le système linéaire précédent, pour chaque ligne j de X , on a :

$$x_{j,1} * S_{i,1} + \dots + x_{j,N} * S_{i,N} = k_{i,j} \Leftrightarrow x_{j,1} * S_{i,1} + \dots + x_{j,N-1} * S_{i,N-1} = k_{i,j} - x_{j,N} * S_{i,N}$$

On peut donc reconstruire une autre matrice X de taille $(N - 1) \times (N - 1)$ puis poser le système linéaire suivant :

$$\underbrace{\begin{bmatrix} x_{1,1} & \dots & x_{1,N-1} \\ x_{2,1} & \dots & x_{2,N-1} \\ \vdots & \vdots & \vdots \\ x_{N-1,1} & \dots & x_{N-1,N-1} \end{bmatrix}}_{(N-1) \times (N-1)} \cdot \underbrace{\begin{bmatrix} S_{i,1} \\ S_{i,2} \\ \vdots \\ S_{i,N-1} \end{bmatrix}}_{(N-1) \times 1} = \underbrace{\begin{bmatrix} k_{i_1} - x_{1,N} \cdot S_{i,N} \\ k_{i_2} - x_{2,N} \cdot S_{i,N} \\ \vdots \\ k_{i_{N-1}} - x_{N-1,N} \cdot S_{i,N} \end{bmatrix}}_{(N-1) \times 1}$$

FIGURE 4 – Représentation du nouveau système linéaire $X \cdot \vec{S}_i^T = \vec{k}_i$, où $k_{i,j} = \vec{S}_i \cdot \vec{x}_j$

Ici, l'ensemble des vecteurs $\{\vec{x}_j\}_{1 \leq j \leq N-1}$ forme une famille libre, donc $\text{rang}(X) = N - 1$, donc X est inversible, donc ce système linéaire possède une unique solution. Cependant, cette unique solution est calculée en fonction de $S_{i,N}$.

En répétant ce procédé M fois, on peut donc réécrire chaque ligne S_i de S en fonction d'un élément $S_{i,N}$.

Cette alternative permet donc de reconstruire chaque élément $S_{i,j}$ ligne de la matrice S en fonction d'un seul élément $S_{i,N}$. Ceci permet à l'attaquant, à chaque requête envoyer à l'oracle, de retrouver une valeur de $S_{i,N}$, et de garder les valeurs de $S_{i,N}$ qui se répètent le plus comme la vraie valeur de $S_{i,N}$.

Remarque : Le choix d'utiliser un élément $S_{i,N}$ pour écrire chaque ligne S_i de S est arbitraire. On aurait pu réécrire chaque ligne S_i de S avec n'importe quel élément $S_{i,l}$. Dans ce cas, dans le système linéaire précédent, chaque ligne $\vec{x}_i$ de X sera $(x_{i,1}, \dots, x_{i,l-1}, x_{i,l+1}, \dots, x_{i,N})$, $\forall i \neq l$, le vecteur $\vec{S}_i^T$ sera $(S_{i,1}, \dots, S_{i,l-1}, S_{i,l+1}, \dots, S_{i,N})$ et chaque élément de $\vec{k}_i$ sera $k_{i,j} - x_{j,l} \cdot S_{i,l}$, $\forall i \neq l$

Cas particulier : On remarque que lorsque l'attaquant envoie un vecteur $\vec{y} = (0, \dots, 0, 1)$ pour recevoir la clé maîtresse $S_{\vec{y}}$, il remarque que $\{(1, 0, \dots, 0), (0, 1, 0, \dots, 0), \dots, (0, \dots, 0, 1, 0)\}$ sont $N - 1$ vecteurs de $(\mathbb{Z}/p\mathbb{Z})^N$ qui sont tous orthogonaux avec $\vec{y}$.

Pour reconstruire la matrice $X \in (\mathbb{Z}/p\mathbb{Z})^{(N-1) \times (N-1)}$, il peut alors choisir un vecteur $\vec{x}_j = (0, \dots, 0, 1, 0, \dots, 0) \in (\mathbb{Z}/p\mathbb{Z})^{N-1}$, pour la ligne j , où on a un 1 au $j^{\text{ième}}$ élément du vecteur.

Ainsi, dans le système linéaire précédent, on a $X = Id_{N-1}$ et $\forall j, x_{j,N} = 0$. On se retrouve donc avec le système linéaire $X \cdot \vec{S}_i^T = \vec{k}_i \Leftrightarrow Id_{N-1} \cdot \vec{S}_i^T = \vec{k}_i$. Ceci permet de retrouver les $N - 1$ premières colonnes de S immédiatement. En effet, dans ce cas particulier, $\forall j \leq N - 1, S_{i,j} = k_{i,j} = \vec{S}_{\vec{y}_i} \cdot \vec{x}_j = S_{\vec{y}_i,j}$.

Donc, après avoir demandé l'évaluation d'un vecteur $EVAL(\vec{x}) = \vec{y}$ à l'oracle, pour retrouver $S_{i,N}$, on a $S_{i,1} \cdot x_1 + \dots + S_{i,N-1} \cdot x_{N-1} + S_{i,N} \cdot x_N = y_i$, donc on a $S_{i,N} = x_N^{-1} \cdot (y_i - S_{i,1} \cdot x_1 - \dots - S_{i,N-1} \cdot x_{N-1})$

3.1.4 Jeu de sécurité pour l'attaque sur F^{WEAK}

Jeu de sécurité sur la CPRF F^{WEAK}

L'attaquant $\mathcal{A}$ interagit avec un oracle O qui répond soit avec une CPRF soit avec une fonction aléatoire. L'objectif de l'attaquant est de distinguer les deux cas.

Phase 1 - Initialisation :

- O construit une clé maîtresse $S \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$.

Phase 2 - Demande de clé contrainte :

- $\mathcal{A}$ demande une clé contrainte en envoyant $\vec{y} = (0, \dots, 0, 1)$ pour recevoir $S_{\vec{y}}$.
- O renvoie $S_{\vec{y}} = S + d \cdot \vec{y}^T$.

Phase 3 - Reconstruction partielle :

- $\mathcal{A}$ calcule $S_{\vec{y}} \cdot \vec{x}$ pour chacun des $\vec{x}$ de chaque ligne de X , pour retrouver k_i , ce qui lui permet de retrouver $S_{i,j}, \forall j \leq N-1$.

Phase 4 - Initialisation du dictionnaire :

- $\mathcal{A}$ initialise un dictionnaire D vide.

Phase 5 - Requêtes d'évaluation et attaque :

- $\mathcal{A}$ peut ensuite demander l'évaluation de plusieurs $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ non orthogonaux à $\vec{y}$ comme suit :
 - Choisir $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ aléatoire en assurant que $x_N \neq 0$ pour assurer que $\vec{x}$ n'est pas orthogonal à $\vec{y}$.
 - Demander à O l'évaluation de $\vec{x} : Eval(\vec{x})$.
 - Calculer $S_{i,N} \forall i$, $\mathcal{A}$ peut le calculer car il connaît les valeurs de $S_{i,1}, \dots, S_{i,N-1}$.
 - Si $(S_{1,N}, \dots, S_{N,N})$ n'est pas dans D , alors insérer dans $D : ((S_{1,N}, \dots, S_{N,N}), 0)$, sinon, incrémenter $D(S_{1,N}, \dots, S_{N,N})$
 - Si $D(S_{1,N}, \dots, S_{N,N}) = \max\{D(\cdot)\}$, alors renvoyer à l'oracle CPRF, sinon, renvoyer fonction aléatoire.

Phase 6 - Interprétation :

Dans ce jeu de sécurité, à chaque demande d'évaluation d'un vecteur $\vec{x}$, l'attaquant recalcule les valeurs de $S_{1,N}, \dots, S_{N,N}$. Les valeurs redondantes ont une haute probabilité que ce sont les vraies valeurs de $S_{1,N}, \dots, S_{N,N}$, ce qui permet d'en déduire que le résultat de l'évaluation vient d'une CPRF. Si après une évaluation, on retrouve des valeurs de $S_{1,N}, \dots, S_{N,N}$ qui ne sont pas redondantes, alors c'est très probablement le résultat d'une fonction aléatoire.

Ceci permet à un attaquant $\mathcal{A}$ de distinguer une CPRF d'une fonction aléatoire, ce qui montre que F^{WEAK} n'est pas sécurisé.

3.1.5 Renforcement de F^{WEAK} par hachage

Afin d'améliorer la sécurité de la CPRF F^{WEAK} , il est possible de faire une implémentation en utilisant une fonction de hachage H . Cette implémentation apporte une modification sur l'évaluation d'une entrée contrainte ou non, sans empêcher un attaquant de demander une clé contrainte.

Évaluation sur les entrées non contraintes : Lorsqu'un attaquant demande l'évaluation d'un vecteur $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, alors l'oracle lui enverrait $Eval_H(\vec{x}) = H(S \cdot \vec{x})$ au lieu de $Eval(\vec{x}) = S \cdot \vec{x}$, où H est une fonction de hachage.

Demande d'une clé contrainte : La demande de la clé contrainte par un attaquant reste inchangée. L'attaquant envoie un vecteur $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^N$, puis l'oracle renvoie $Constrain(S, \vec{y}) = S_{\vec{y}} = S + d \cdot \vec{y}^T$.

Évaluation sur les entrées contraintes : Après avoir reçu une clé contrainte S_y , pour évaluer un vecteur $\vec{x}$ orthogonal à $\vec{y}$, l'attaquant calculera $Eval_C H(\vec{x}) = H(S_y \cdot \vec{x})$ au lieu de $Eval_C(\vec{x}) = S_y \cdot \vec{x}$, où H est une fonction de hachage.

Cette implémentation qui utilise une fonction de hachage H , assure qu'un attaquant puisse toujours faire des évaluations sur les entrées contraintes, mais il ne pourra pas reconstituer la matrice S en résolvant des systèmes linéaires. En effet, en utilisant l'attaque décrite dans la section précédente, retrouver S revient à chercher une pré-image $H(S \cdot \vec{x})$. Ceci confirme que l'application d'une fonction de hachage assure la sécurité de la CPRF F^{WEAK} .

3.1.6 Preuve de sécurité de F^{WEAK} hachée

Théorème 2 (CJ25 [2])

Soit CF une CPRF. Si les conditions suivantes sont satisfaites :

- $\min_x |\text{CF.Out}(\lambda, x)|$ est super-polynomial en λ ,
- CF est IND*-NC-CPRF sécurisée,
- L'oracle aléatoire utilisé est non-programmable,

alors la construction $\text{Hashed}[CF]$ est **SIM*-AC-CPRF sécurisée**, c'est-à-dire qu'aucun adversaire efficace (à nombre polynomial de requêtes) ne peut distinguer cette construction d'un idéal, même en présence d'un oracle de hachage aléatoire.

Fonction super-polynomiale

Une fonction $f : \mathbb{N} \rightarrow \mathbb{N}$ est dite *super-polynomiale* si, pour tout polynôme $p(\lambda)$, il existe N tel que pour tout $\lambda \geq N$,

$$f(\lambda) > p(\lambda).$$

Autrement dit, $f(\lambda)$ croît plus vite que toute fonction polynomiale.

- $\min_x |\mathbf{F}^{\text{WEAK}}.\text{Out}(\lambda, x)|$ est super-polynomial en λ :

Dans l'implémentation de la CPRF F^{WEAK} , nous travaillons avec des coefficients de $\mathbb{Z}/p\mathbb{Z}$ où p est de taille λ . On a donc $|\mathbb{Z}/p\mathbb{Z}| \approx 2^\lambda$.

L'évaluation d'une entrée $x \in (\mathbb{Z}/p\mathbb{Z})^N$ par F^{WEAK} se fait par $\text{Eval}(F^{\text{WEAK}}, \vec{x}) = S \cdot \vec{x}$, ce qui est une application linéaire, qui est donc une bijection. Ceci implique que $\mathbf{F}^{\text{WEAK}}.\text{Out}(\lambda, x)$ a la même taille pour tout $x \in (\mathbb{Z}/p\mathbb{Z})^N$.

Fixons $x \in (\mathbb{Z}/p\mathbb{Z})^N$. Pour tout $S, S' \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$, $S \neq S' \Rightarrow S \cdot x \neq S' \cdot x$. Comme S est de taille $M \times N$, et que les coefficients sont dans $\mathbb{Z}/p\mathbb{Z}$, on en déduit qu'il existe $(2^\lambda)^{M \times N} = 2^{\lambda \times M \times N}$ clé maîtresse différentes.

On conclue que $\min_x |\mathbf{F}^{\text{WEAK}}.\text{Out}(\lambda, x)| \approx 2^{\lambda \times M \times N}$ qui est super-polynomial en λ .

- L'oracle aléatoire utilisé est non-programmable :

On rappelle que la fonction lazy sampling $P.Ls$ et la fonction de programmation $P.Prog$ sont implémentées comme suit :

$$\begin{array}{l|l} P.Ls(1^\lambda, x, \sigma_P) & P.Prog(1^\lambda, x, y, \sigma_P) \\ \text{SI } \sigma_P[x] = \perp \text{ alors } \sigma_P[x] \leftarrow \text{Random}(1^\lambda) & \text{Si } \sigma_P[x] = \perp \text{ alors } \sigma_P[x] \leftarrow y \\ \text{Renvoyer } \sigma_P[x] & \text{Renvoyer } \perp \end{array}$$

On dit que P est non programmable si $P.Prog$ ne met pas à jour σ_P et l'exécution de $P.Ls$ sur différentes entrées ne change pas la distribution de sa sortie.

Dans notre implémentation de F^{WEAK} , nous utilisons le lazy sampling pour implémenter la fonction de hachage H . Ceci implique que $\text{Eval}_H(\vec{x}) = H(S \cdot \vec{x})$ est fixé uniquement lorsque l'évaluation de $\vec{x}$ est demandée, mais pas plus tôt.

On peut donc conclure que P est non-programmable.

- F^{WEAK} est IND*-NC-CPRF sécurisée :

Pour cette condition, on peut prouver que CF est IND*-NC-CPRF sécurisée si il n'existe pas un attaquant qui :

- Demande toutes les évaluations
- Effectue ses calculs
- Renvoie ses réponses à l'oracle tout en ayant un avantage significatif

Dans notre jeu de sécurité, dans la phase 5, l'attaquant procède comme suit :

- Demande **une** évaluation
- Effectue ses calculs
- Renvoie sa réponse à l'oracle tout en ayant un avantage significatif

— Recommence

Cependant, il est possible de modifier la phase 5 de notre jeu de sécurité pour que l'attaquant procède comme suit :

- Choisir n vecteurs $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ aléatoire en assurant que $x_N \neq 0$ pour chacun des vecteurs $\vec{x}$ pour assurer que $\vec{x}$ n'est pas orthogonal à $\vec{g}$.
- Demander à O l'évaluation des n vecteurs $\vec{x} : Eval(\vec{x})$.
- Pour chaque évaluation des n vecteurs $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, calculer $S_{i,N} \forall i$.
- Pour chaque $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, si $(S_{1,N}, \dots, S_{N,N})$ n'est pas dans D , alors insérer dans $D : ((S_{1,N}, \dots, S_{N,N}), 0)$, sinon, incrémenter $D(S_{1,N}, \dots, S_{N,N})$
- Pour chaque $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, si $D(S_{1,N}, \dots, S_{N,N}) = \max\{D(\cdot)\}$, alors renvoyer à l'oracle CPRF, sinon, renvoyer fonction aléatoire.

Un attaquant aura les mêmes avantages en utilisant cet attaque, sans effectuer des évaluations adaptatives, ce qui montre que F^{WEAK} n'est pas IND*-NC-CPRF sécurisée.

Toutefois, le fait que F^{WEAK} n'est pas IND*-NC-CPRF sécurisée n'implique pas que $\text{Hashed}[F^{\text{WEAK}}]$ n'est pas **SIM*-AC-CPRF sécurisée**. Ceci n'est donc pas une preuve que $\text{Hashed}[F^{\text{WEAK}}]$ n'est pas **SIM*-AC-CPRF sécurisée**.

3.2 Permutation à trappe de Rabin

La permutation de Rabin est une fonction à trappe fondée sur la difficulté de la factorisation des entiers. Elle repose sur l'opération de mise au carré modulo un entier composé.

— **Génération de la clé :**

On choisit deux nombres premiers distincts p et q tels que :

$$p \equiv q \equiv 3 \pmod{4}$$

La clé publique est définie par :

$$n = p \cdot q$$

La clé privée est le couple (p, q) .

Cette condition sur p et q permet un calcul efficace des racines carrées modulo ces nombres premiers.

— **Permutation de Rabin :**

La permutation de Rabin est la fonction :

$$f : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$$

définie par :

$$f(x) = x^2 \pmod{n}$$

Cette fonction est facile à calculer, mais difficile à inverser sans connaître la factorisation de n .

— **Inverser la permutation de Rabin :**

Inverser la permutation de Rabin revient à résoudre l'équation :

$$x^2 \equiv y \pmod{n}$$

Lorsque $n = p \cdot q$, cette équation admet exactement **quatre solutions distinctes** modulo n .

On commence par réduire le problème modulo p et q :

$$\begin{cases} x^2 \equiv y \pmod{p} \\ x^2 \equiv y \pmod{q} \end{cases}$$

Comme $p \equiv 3 \pmod{4}$, une racine carrée modulo p est donnée par :

$$r_p = y^{\frac{p+1}{4}} \pmod{p}$$

De même, modulo q :

$$r_q = y^{\frac{q+1}{4}} \pmod{q}$$

Les solutions sont donc :

- $X_0 = (r_p * q * q_{\text{mod } p}^{-1} + r_q * p * p_{\text{mod } q}^{-1}) \bmod n$
- $X_1 = (r_p * q * q_{\text{mod } p}^{-1} - r_q * p * p_{\text{mod } q}^{-1}) \bmod n$
- $X_2 = (-r_p * q * q_{\text{mod } p}^{-1} + r_q * p * p_{\text{mod } q}^{-1}) \bmod n$
- $X_3 = (-r_p * q * q_{\text{mod } p}^{-1} - r_q * p * p_{\text{mod } q}^{-1}) \bmod n$

Problème : Laquelle de ces racines correspond au message original ?

La fonction $f(x) = x^2 \bmod n$ n'est pas injective : chaque chiffré c a 4 pré-images. Même avec la clé secrète (p, q) , on peut calculer toutes les racines mais **pas déterminer directement celle qui correspond au message initial**. Appliquer l'inverse $n - c$ fois n'assure pas de retrouver le message original.

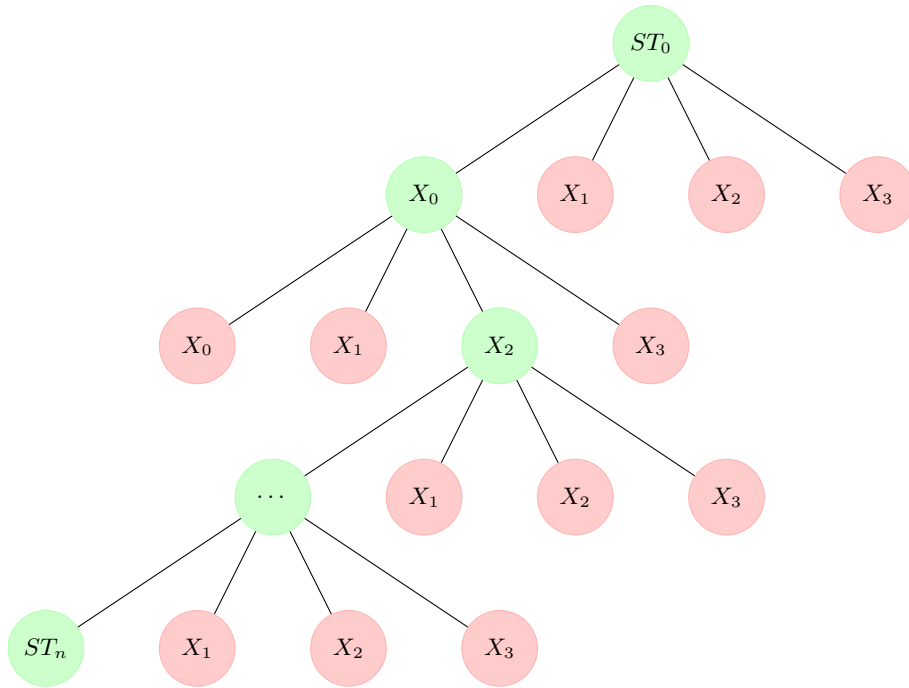


FIGURE 5 – Arbre des pré-images dans la permutation de Rabin.

Conclusion : La permutation de Rabin n'est pas adaptée pour notre CPRF car chaque valeur a quatre pré-images possibles. Même avec la clé privée, il est impossible de savoir laquelle correspond à l'entrée originale, ce qui empêche l'application répétée de l'inverse $(n - c)$ fois pour retrouver correctement le résultat de la CPRF.

3.3 Permutation à trappe AES

Le chiffrement AES applique une fonction bijective $E_k(x)$ qui dépend de la clé k . Contrairement à $x \mapsto x^2 \bmod n$, chaque bloc chiffré a une **unique pré-image** pour une clé donnée. Cependant, pour déchiffrer, **il faut absolument connaître la clé k** :

- Sans la clé, l'attaquant ne peut pas inverser la permutation.
- AES reste sécurisé grâce à la clé secrète.

Donc l'utilisation d'AES comme permutation à trappe n'est pas adaptée pour notre CPRF.

4 L'attaque CJ25

4.1 Description générale et intuition de l'attaque

L'attaque CJ25 exploite le fait qu'un attaquant peut obtenir une clé contrainte permettant d'évaluer la fonction pseudo-aléatoire sur un sous-ensemble d'entrées, tout en conservant l'accès à un oracle d'évaluation global.

Dans un premier temps, l'adversaire choisit un indice n et demande à l'oracle une clé contrainte (PK, ST_n, n) . Cette clé lui permet d'évaluer la fonction sur toute entrée x telle que $x < n$ à l'aide des permutations publiques associées.

Dans un second temps, l'adversaire interroge l'oracle d'évaluation sur une entrée $x < n$ et obtient une valeur ST_x . À ce stade, l'attaquant ne sait pas si ST_x provient d'une fonction pseudo-aléatoire contrainte ou d'une fonction aléatoire.

À l'aide de la clé contrainte, l'attaquant applique successivement les permutations publiques afin de calculer la valeur $\pi_{PK}^{x-n}(ST_x)$. Si l'attaquant retrouve $ST_a = ST_x$, il peut alors conclure que la valeur ST_x est le résultat d'une fonction pseudo-aléatoire contrainte.

Il est donc possible pour un attaquant d'avoir des informations sur la structure de la CPRF à partir de la clé contrainte et des évaluations obtenues, ce qui lui permet de distinguer une CPRF d'une fonction aléatoire avec une probabilité non négligeable.

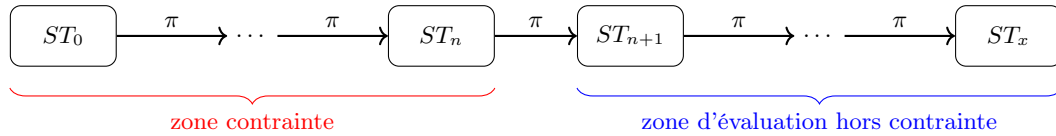


FIGURE 6 – Schéma général de l'attaque CJ25

4.2 Modélisation du jeu de sécurité et avantage de l'attaquant

Jeu de sécurité pour les CPRF

L'attaquant $\mathcal{A}$ interagit avec un oracle $\mathcal{O}$ qui est soit une *CPRF* soit une fonction aléatoire. L'objectif de l'attaquant est de distinguer les deux cas.

- **Phase 1** : $\mathcal{A}$ demande une clé contrainte $K_C = (PK, ST_n, n)$ pour une contrainte n de son choix, lui permettant d'évaluer la fonction sur un sous-ensemble d'entrées.
- **Phase 2** : $\mathcal{A}$ fait un nombre de requêtes d'évaluation à l'oracle $\mathcal{O}$ sur des entrées $x > n$ pour obtenir des valeurs ST_x non contraintes.
- **Phase 3** : $\mathcal{A}$ utilise les valeurs obtenues pour évaluer $ST_a = \pi_{PK}^{n-x}(ST_x)$ et compare avec ST_n .
- **Décision** : $\mathcal{A}$ décide que $\mathcal{O}$ est la *CPRF* si $ST_a = ST_n$, sinon il décide que $\mathcal{O}$ est une fonction aléatoire.

Dans le cas où $\mathcal{O}$ est une *CPRF*, l'attaquant $\mathcal{A}$ retrouve toujours $ST_a = ST_n$, et il ne se trompe jamais. En revanche, si $\mathcal{O}$ est une fonction aléatoire, la probabilité que $ST_a = ST_n$ est de $1/N$, où N est la taille de l'espace de sortie de la fonction. Dans ce cas, l'attaquant se trompe avec une probabilité de $1/N$. L'attaquant $\mathcal{A}$ gagne donc avec une probabilité de $1 - 1/N$ de distinguer correctement les deux cas, ce qui est non négligeable pour des tailles de N raisonnables.

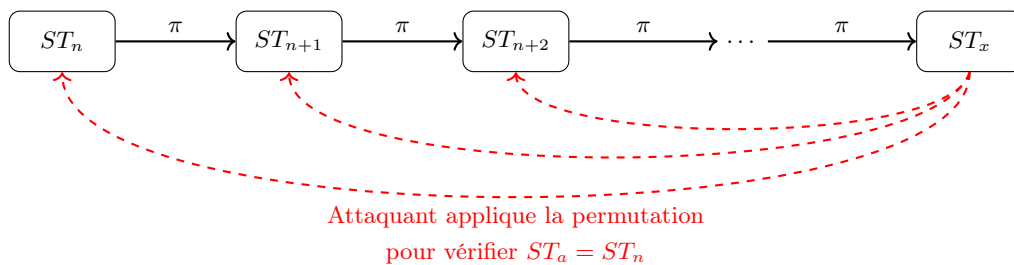


FIGURE 7 – Schéma des permutation de l'attaque dans la zone hors contrainte

4.3 Mise en œuvre expérimentale de l'attaque

Pour simuler le jeu de sécurité, nous avons mis en place une architecture client-serveur locale reposant sur TCP. Le serveur implémente l'oracle du jeu (évaluation et génération de clés contraintes), le client joue le rôle de l'attaquant (requêtes d'évaluation et de contrainte, tests, ...).

4.3.1 Implémentation de l'oracle de sécurité

L'oracle met à disposition deux interfaces accessibles à l'attaquant :

- **EVAL(x)** : retourne une valeur associée à l'entrée x .
En monde PRF, l'oracle retourne l'évaluation de la CPRF à l'aide de la clé maîtresse.
En monde aléatoire, il retourne une valeur uniforme de même taille.
- **CONSTRAIN(n)** : retourne une clé contrainte associée à l'indice n , permettant l'évaluation de la fonction sur un sous-ensemble d'entrées.

4.3.2 Implémentation de l'attaquant et stratégie

L'attaquant choisit un indice n et demande à l'oracle la clé contrainte correspondante. Cette clé lui permet d'évaluer la fonction sur des entrées strictement supérieures à n .

Ensuite il effectue des requêtes **EVAL** sur des entrées $x > n$. À partir des valeurs retournées par l'oracle et de la clé contrainte, il applique les permutations publiques afin de tester la cohérence de la structure de la CPRF.

Si l'attaquant trouve une correspondance, il peut conclure que l'oracle implémente une CPRF plutôt qu'une fonction aléatoire.

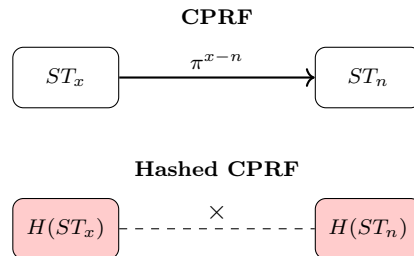
5 Renforcement de la CPRF par hachage

Afin d'améliorer la sécurité de la construction de BMO17 face à l'attaque CJ25, il est possible d'implémenter une version hachée de la CPRF.

5.1 Principe général de la CPRF hachée

L'idée est d'introduire une fonction de hachage cryptographique dans l'évaluation. L'attaquant pourra toujours évaluer la CPRF sur les entrées non contraintes, mais il ne pourra pas exploiter les résultats pour distinguer la CPRF d'une fonction aléatoire car ceux-ci seront hachés de manière irréversible.

→ Le rôle de la fonction de hachage est de rendre impossible toute utilisation ou comparaison via les permutations publiques.



Implémentation :

Nous avons implémenté deux versions de la CPRF hachée :

- **CPRF hachée simple** : l'évaluation est effectuée normalement, puis le résultat est haché à l'aide de **SHA-256** avant d'être retourné.
 - $Eval_H(x)$: retourne le hachage de l'évaluation sur l'entrée x avec la clé maîtresse, c'est-à-dire $H(Eval((SK, ST_0), x))$.
 - $EvalC_H(n)$: retourne le hachage de l'évaluation sur l'entrée n avec la clé contrainte, c'est-à-dire $H(EvalC((PK, ST_n, n), c))$.
- **CPRF hachée avec lazy sampling** : afin de se rapprocher du modèle théorique d'un oracle de hachage idéal utilisé dans l'article CJ25, nous avons également implémenté une version reposant sur le lazy sampling décrite dans la section suivante.

5.2 Construction explicite de la CPRF hachée de Cheng et Jaeger

Nous présentons ici la version hachée proposée dans l'article CJ25.

Notations utilisées :

- P : fonction de hachage idéale modélisée comme un oracle de hachage aléatoire.

- σ_P : état interne de P (initialement vide) qui mémorise les paires (x, y) .
- λ : paramètre de sécurité (taille de la sortie de P).
- k : clé secrète de la CPRF.
- k_R : clé contrainte associée à un ensemble de restrictions R .
- x : entrée de la fonction.
- $y \in \{0, 1\}^\lambda$: valeur de hachage associée à une entrée.

L'évaluation est définie par :

$$\begin{aligned} \text{Hashed[CPRF].Eval}(1^\lambda, k, x) &= P.\text{LazySampling}(1^\lambda, \text{CPRF.Eval}(1^\lambda, k, x) : \sigma_P). \\ \text{Hashed[CPRF].EvalC}(1^\lambda, k_R, R, x) &= P.\text{LazySampling}(1^\lambda, \text{CPRF.EvalC}(1^\lambda, k_R, R, x) : \sigma_P). \end{aligned}$$

Lazy Sampling

Chercher x dans σ_P
 Si $x \notin \sigma_P$:
 Choisir $y \leftarrow \{0, 1\}^\lambda$ au hasard
 Mémoriser (x, y)
 Sinon :
 Récupérer y associé à x
 Retourner y

Algorithme de lazy sampling d'une primitive idéale.

6 Résultats expérimentaux

Après avoir implémenté différentes variantes de la CPRF (version originale, version hachée avec SHA-256 et version hachée avec lazy sampling), nous avons mené des expériences afin d'évaluer l'efficacité de l'attaque CJ25 sur chacune d'elles. Les tests consistent en l'exécution de 100 requêtes d'évaluation par version, et en l'observation de l'évolution du taux de succès de l'attaque au fil des requêtes.

La figure 8 présente les résultats obtenus sur la version non hachée de la CPRF. On observe que l'attaque CJ25 est très efficace dans ce cas, avec une probabilité de succès proche de 1 dès les premières requêtes. Cela s'explique par le fait que la structure de la CPRF reste entièrement exploitable dans ce modèle, ce qui permet à l'attaquant de distinguer les comportements de la fonction.

Dans les figures 9 et 10, la même attaque est appliquée aux versions hachées (SHA-256 et lazy sampling). On observe alors un échec systématique de l'attaque, avec un taux de succès proche de 0.5, correspondant à un comportement équivalent à une décision aléatoire. Dans ces cas, l'attaquant ne parvient plus à exploiter la structure de la CPRF.

Enfin, les résultats suggèrent que le choix du schéma de hachage (SHA-256 ou lazy sampling) n'a pas d'impact significatif sur la robustesse observée dans nos expérimentations, les deux variantes présentant des comportements similaires face à l'attaque CJ25.

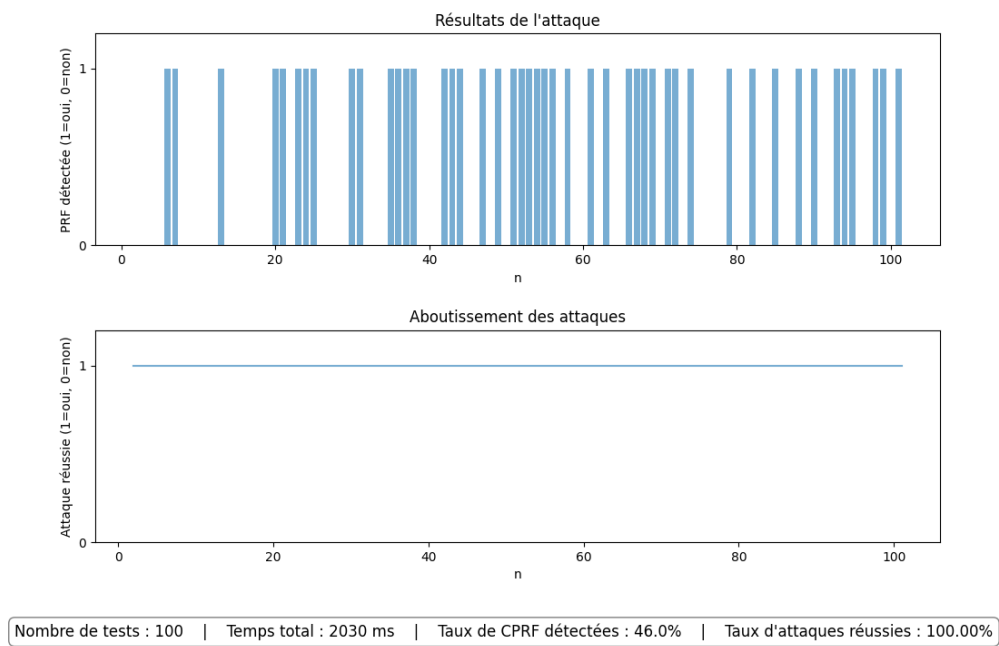


FIGURE 8 – Attaque version non hachée

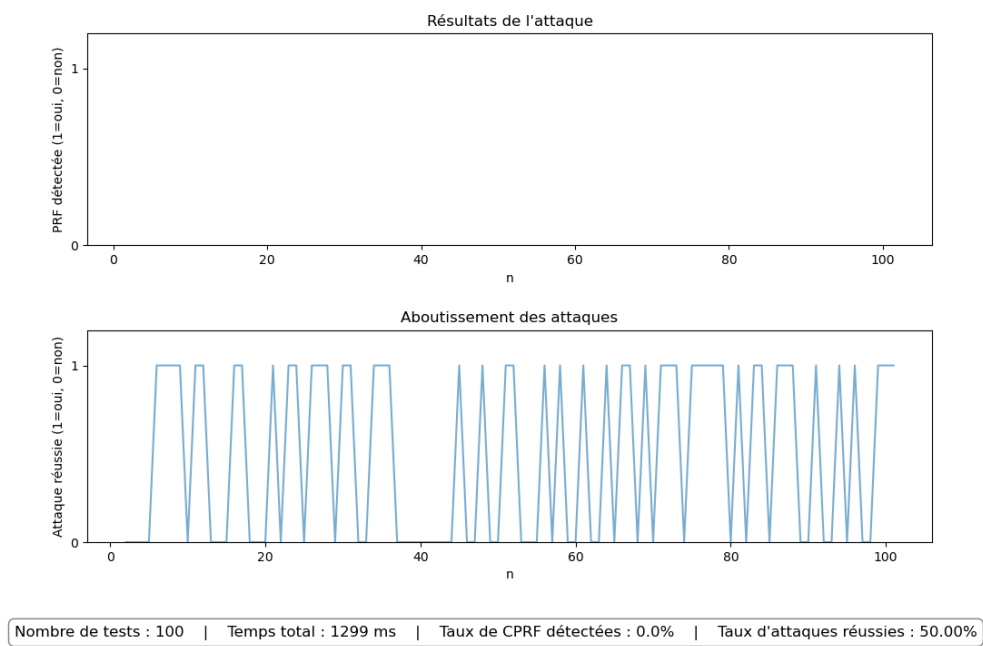


FIGURE 9 – Attaque version hachée avec SHA-256

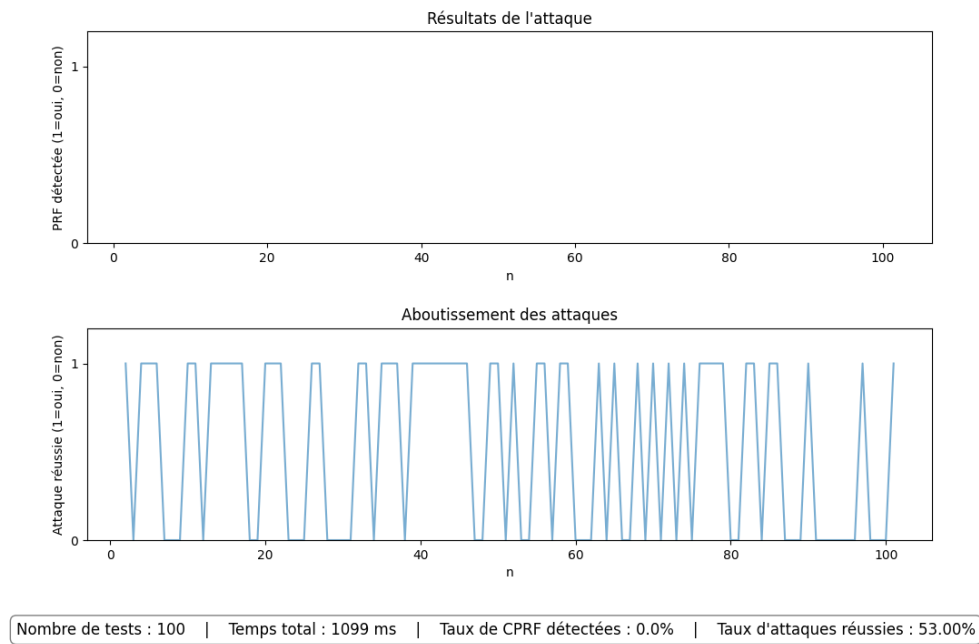


FIGURE 10 – Attaque version hachée avec lazy sampling

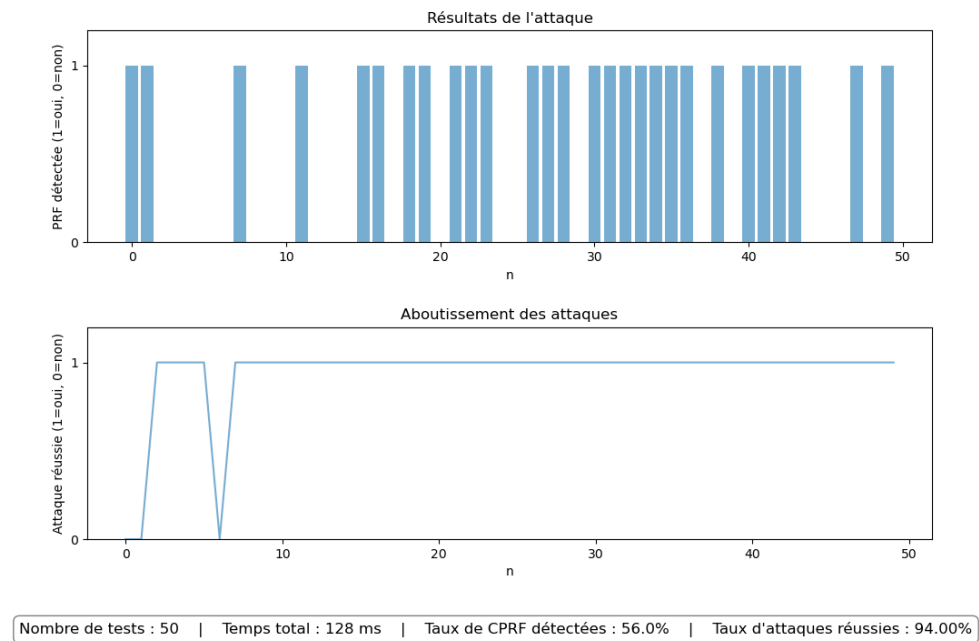


FIGURE 11 – Attaque Fweak

Références

- [1] E. Bost, B. Minaud, and O. Ohrimenko, *Forward and Backward Private Searchable Encryption from Constrained Cryptographic Primitives*, ACM CCS 2017.
- [2] K. Cheng and J. Jaeger, *Adaptive Security for Constrained PRFs*, 2025.